

Tuesday, 3rd Feb 2026.

Data Structure

→ Abstract data types

→ Algorithm

→ Arrays

→ Stack (FILO)



→ Every time access the top first in a stack

→ Applications (LIFO) of a stack

→ Infix to Postfix Conversion

→ Postfix Evaluation

→ Queue (FIFO)

(1) Linear Queue

(2) Circular Queue

(3) Double Ended Queue

(4) Priority Queue

• Applications of Queue ::

(All above are static data structure)

→ Linked List → Pointers

(dynamic data structure)

1) Linear linked list

2) Circular linked list

3) Doubly linked list

4) Doubly circular linked list

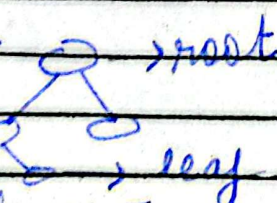
→ Trees

Binary Trees

BST

Height Balance Trees → AVL

Heap Tree (4)



After mid

→ Heap Tree

→ Searching

Linear
Binary

Sorting

Bubble sort

Insertion sort

Selection sort

Merge sort

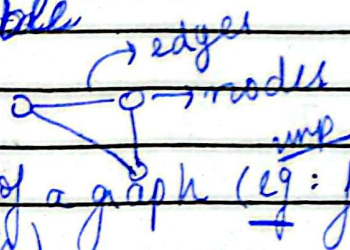
Quick sort

→ Hashing

↳ Hash functions

↳ Hash table

→ Graphs



Applications of a graph (eg: find the shortest path)

→ Time complexity

$O(n)$

$O(n^2)$

$\log(n)$

Book:-

* Data Structures in C++ by Erro and Temmen
* by Mark Alan

Factors to write down good algorithm:

- ① Processing Time to execute the problem
(less time = better algorithm)
- ② Memory (RAM) (less memory = good algo)
- ③ Accuracy eg: 5798
- ④ Sequence (order) 1) 5378
- ⑤ General ✓ 2) 559.8 more accurate algo near to it
- ⑥ Language Independent
- ⑦ Termination Criteria

ARRAYS: collection of same type of data in a contiguous (continuous) memory location

2 Types:

1) Linear Array

2) 2D array

`int a[rows][cols];`

`int a[3][3];`

↓
 $4 \times 9 = 36 \text{ bytes}$

index: $3 \times 3 = 9 \text{ elements}$

0	1	2	3
1	4	5	6
2	7	8	9

`int a[10];` 
a = 0 to 9
index

→ Row Major order:

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

1st row → 2nd row → 3rd row

→ Column major order:

1	4	7	2	5	8	3	6	9
---	---	---	---	---	---	---	---	---

1st col → 2nd col → 3rd col

Feb 12, 2026

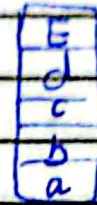
Thursday

STACK: "is an ordered collection of elements in which elements may be inserted or deleted at one end that is called the **Top** end of the stack."

LIFO / FILO

• Insertion $\rightarrow$ Push

Deletion $\rightarrow$ Pop

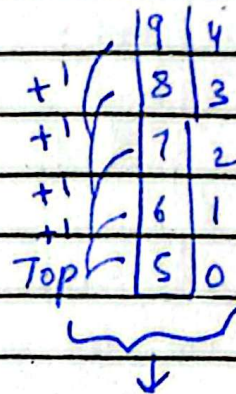


Memory Representation of stack :-

Array, Linked list

Stack starts $\downarrow$

```
{ int stack[5];  
    index: 0-4  
    int Top; }
```



Stack S;

S stack [S.TOP];

Top = -1;

$$\begin{aligned} \text{Top} &= -1 + 1 = 0 + 1 = 1 \\ &= 1 + 1 = 2 \end{aligned}$$

PUSH

Algorithm for Push

1. Overflow

Yes No
 $\downarrow$ $\downarrow$
return

2. Increment by 1 in Top.

3. Add/Insert value on top of the stack

$$5 - 1 = 4$$

Code if (Top == size() - 1) {
 cout << "overflow";
 return;
} else {
 Top++;
 stack[Top] = Value;

eg: Stack[4];

10	3
4	2
8	1
7	0

Top = -1 (True)

If (Top == size - 1)
 cout << "overflow";
 return;

else {
 Top++;
 stack[Top] = value;
}

POP

• Algorithm:

1. Underflow
 Yes NO
 return

10	3
0	2
0	1
0	0

0 - 1 = -1
True loop ends

2. Store Top of stack in some temporary variable.

3. Assign null / 0 to the top of the stack.

4. Decrement by 1 in top.

• Code: If (Top == -1) {
 cout << "Underflow";
 return;}
else {
 temp = stack[Top];
 stack[Top] = 0;
 Top--;
 stack[Top];
}

Q Stack[4]

Push(0) overflow
 Push($3 + 5 \times 11$) overflow
 Push($x + 9$) 22
 Push(x/y) 0
 $y = \text{pop}()$ 23
 $x = \text{pop}()$ 13
 Push(y) overflow
 Push($9 - 3 + 7$) 13
 Push($3 \times 7 + 2$) 23
 Push($5 \div 9$) 5
 Push($x + y$) 14

a) Print

- $x = 13$
- $y = 23$

overflow
 Top $\rightarrow$ 3 22 13/23 = 0
 Top $\rightarrow$ 2 2
 pop 5 1
 14 0 $\rightarrow$ Top

- pop and print until stack is empty = 14

16 Feb 2026

Monday

APPLICATIONS OF STACK

1. Basic Conversion Algorithm

• Decimal to Binary $\rightarrow$ divide by 2

Input: Decimal no

Output: Binary representation

• Algorithm

1. Create a stack to hold the top $\rightarrow$ 1
remainders

2. Get the input

3. loop, while (no > 0)

1) calculate the remainder divided by 2 $\text{Rem} = \text{no} \% 2$

2 | 10
 2 | 5 - 0
 2 | 2 - 0
 2 | 1 - 1
 0 - 1
 1010

top $\rightarrow$ 1
 0
 1
 0

1010 stack

1. $\wedge$ (power)
 2. $*, /$
 3. $+, -$
- left to right if comes together

Tuesday, Feb 17, 2026

Data Structures

Applications of Stacks

Infix

Postfix

Prefix

- operator b/w 2 operands
- operator after operands
- operator before operands

- $a + b$
- $ab +$
- $+ ab$

Infix to Postfix

- $a + b * c$
- $(a + b) * (c - d)$

$$a + bc *$$

$$ab + * cd -$$

$$abc * +$$

$$ab + cd - *$$

Infix to Prefix

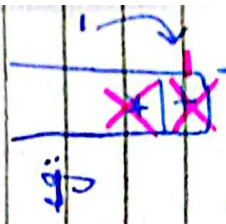
$$a + b * c$$

$$(a + b) * (c - d)$$

$$+ a * bc$$

$$+ ab * - cd$$

$$* + ab - cd$$



Pop and display with postfix

Infix to Postfix conversion using Stack

(1) $a + b * c$

Input

Postfix

Stack

a

a

+

+

+, *

b

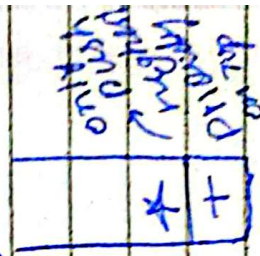
+

+

$$[abc * +]$$

*

empty



(2) $(a + b) * (c - d)$

(

(

a

a

+

(, +

b

$$[ab + cd - *]$$

empty

Note : "(" open = add to stack
 ")" closing = pop until matching opening met

* , (, -
 *
 Empty

- At end of expression stack must be empty otherwise pop elements from stack
- Same priority so pop and add new

$$(3) (A+B) + C - (D-E) \wedge F$$

$AB+C+DE-FA-$	$(, +$
	Empty
	$+ , -$
	$- , \wedge$
	Empty

$$(4) A+B * C / D - F + A \wedge E$$

A	$+, *$
B	$+, /$
$ABC * B / + F - AEA +$	$+$
C	$-$
/	$+, \wedge$
D	
-	
F	
+	
A	
$\wedge$	
E	

$$(4) A - (B + C / D - (E + F * G) - H)$$

Input

Postfix

Stack

-	A	-
(	AB	- , C
B	ABC	- , C , +
+	ABCD / +	- C , *
C	ABCD / + E	- , C , -
D	ABCD / + EF	- , C - , C
-	ABCD / + EFG	- , C - , C , +
(	ABCD / + EFG * +	- , C , -
E	ABCD / + EFG * + -	- , C , -
+	ABCD / + EFG * + - H	- , C , -
F	<u>ABCD / + EFG * + - H - -</u>	empty

③ At end stack only 1 value contains

② pop there 2 values

①

For operator : 2 values should exist before it in stack

Postfix Evaluation using Stack

Let us right

$$(1) 7, 4, -3, *, 1, 5, +, 1, *$$

Stack

op2 op1

$$7, 4, -3$$

$$4 - 3$$

$$7, -12, 1, 5$$

$$4 * -3 = -12$$

$$7, -12, 6$$

$$1 + 5 = 6$$

$$7, -2$$

$$-12 / 6 = -2$$

$$\boxed{-14}$$

$$7 * -2 = -14$$

$$(2) 4, 5, 7, 2, +, -, *$$

Stack

op2 op1

$$4, 5, 7, 2$$

$$7 + 2 = 9$$

$$4, 5, 9$$

$$5 - 9 = -4$$

$$4, -4$$

$$4 * -4 = -16$$

$$\boxed{-16}$$

(3) 4, 12, 3, 5, 1, -, *, +, *, +, *

stack
4, 2, 3, 5, 1 op¹ = 5 op² = 1 = 4

4, 2, 3, 4 3 + 4 = 7

4, 2, 7 2 * 7 = 14

4, 14 4 + 14 = 18

18

Invalid expression bcz * is there and stack is having only 1 value.

Algorithm:-

1. Initialize an empty stack.
2. Check the postfix expression from left to right.
3. For each symbol

→ If operand: push it onto the stack
→ If operator:

- pop the top 2 elements
- Apply the operator **operand 1** operator **operand 2**
- Push result back onto stack

4. After checking, top of stack is, the final result

5. If no two values exist in stack, then it is an invalid expression

Code:- #include <iostream>

#include <cmath>

using namespace std;

const int MAX = 100;

int stack[MAX];

int top = -1;

"Linear list of elements in which elements are inserted at one end and that end is called the rear end of queue, deletion takes place at other front called front and queue"

QUEUE (FIFO)

(1) Linear Queue

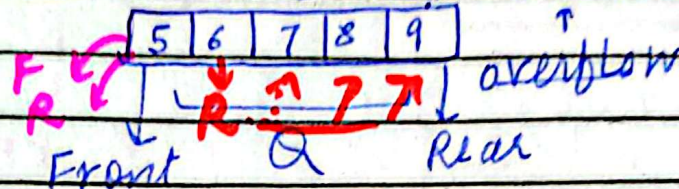
Memory Allocation (Representation) of Linear Queue

struct Queue

```
{
    int Q[5];
```

```
    int F = -1;
```

```
    int R = -1;
}
```



* only when Queue is empty add +1 in F, R
 $F = -1 + 1 = 0$ * otherwise +1 in R only.
 $R = -1 + 1 = 0$

• Insertion:- Algo:

1. overflow (check if space is empty)

Yes NO

return

2. If Queue is empty $\rightarrow$ Yes

• Increment by 1 in F and in R

• Add value at rear end.

$\rightarrow$ NO

• Increment by 1 in R and add value at R end

$\Rightarrow$ 1. If $(F == 0 \ \& \ R == \text{size}() - 1)$

```
    cout << "overflow";
    return;
```

2. else if $(F == R == -1)$

```
    F++;
```

```
    R++;
```

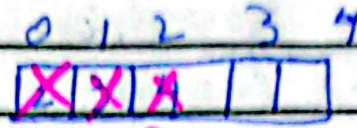
```
    Q[R] = value;
```


3. else

$R++;$

$Q[R] = \text{value};$

Deletion

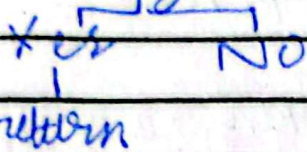


R

F

$F = R = -1$

1. Underflow



2. If F, R at same index / if Q has only one value, then re-initialize with -1

NO-Increment by 1 in front

If deletion value is required save that in temp variable or assign 0 or Null at that index of Q

$\Rightarrow$ 1. If $(F == R == -1)$

cout << "underflow";

return;

2. elseif $(F == R)$

temp = $Q[F];$

$Q[F] = 0;$

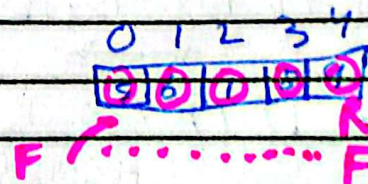
$F = R = -1;$

3. else

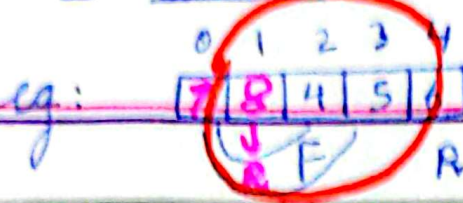
temp = $Q[F];$

$Q[F] = 0;$

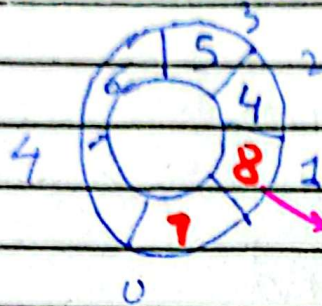
$F++;$



Q. Insertion



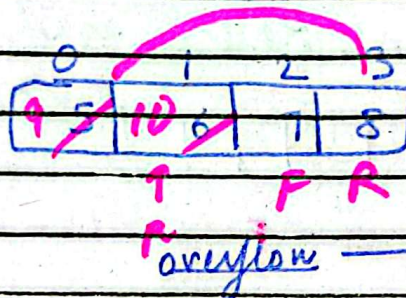
Ans: Shiftback() } $\Rightarrow$ lengthy
we convert it to Circular Queue



$\Rightarrow$ overflow $\Rightarrow$ $F = R + 1$
 $R = 1 + 1$
 $R = 2$

(2) Circular Queue

$F = -1$
 $R = -1$



$\Rightarrow$ $F = 0$ & $R == size - 1$
 $\Rightarrow (F = R + 1)$

* Linear Queue is not used due to memory wastage

Insertion:-

1. Overflow
Yes No
return

1. If $(F == R == -1)$
cout << "overflow"
return;

2. else if $(F == 0 \&\& R == size - 1)$

2. If Queue is empty $\rightarrow$ Yes
• Increment by 1 in R and F
• Add value at rear end

Double Ended Circular

Queue

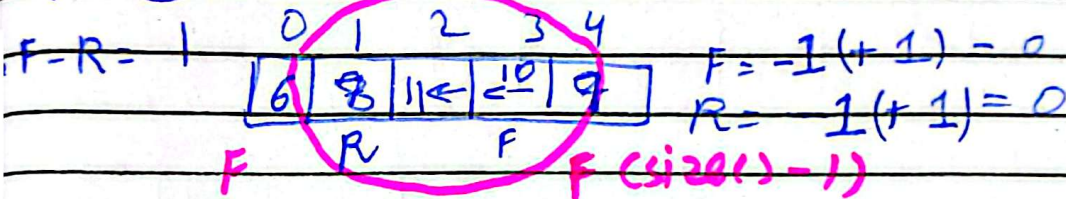
Insert

insert

delete

delete

- 1) Insertion at Rear end
- 2) Deletion at Front end
- 3) Insertion at Front end
- 4) Deletion at Rear end



overflow: $F = R + 1$

1st value = 0 index (F)
2nd value = size()-1 (F)

$3 = 2 + 1$
 $3 = 3$

$F = 0$ & $R = \text{size}() - 1$
 $F = R + 1$

(I) Insertion at Front

1. overflow

if (F == 0 & R == size() - 1) ||
(F == R + 1)

2. else if (F == R == -1)

F++;

R++;

arr[F] = value;

3. else if ($F == 0$)
 $F = \text{size}() - 1;$
 $Q[i] = \text{value};$

else

$F--;$
 $Q[F] = \text{value};$

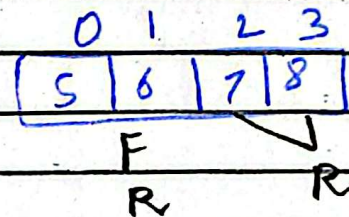
(2) Delete From Rear

1. Underflow

if ($F == R \neq -1$)
 cout << "underflow";

2. else if ($F == R$)

$\text{temp} = Q[R];$
 $Q[R] = 0;$
 $F = R = -1;$

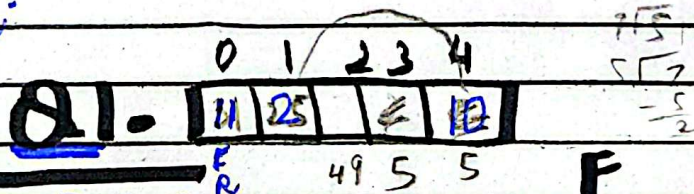


3. else if ($R == 0$)

$\text{temp} = Q[R];$
 $Q[R] = 0;$
 $R = \text{size}() - 1;$

4. else

$R--;$



Insertion at Rear ($3 \times 2 + 5$)
 ($7 \% 5$)

Insertion at Front ($9 \times 3 + 4$)

(25)

(27×2)

overflow = ($4 \% 3 \times 7$)

Delete From Front ()

Delete from Rear ()

Delete from Rear ()

Insertion at Front (5)

" (35. / . 7)

" (49)

"

Insertion at Rear (25. / . 2 - 3)

(2 + 4)

• Delete from Rear until Queue is empty.

Wednesday, Feb 26, 2026. $\rightarrow$ works like simple queue.

(3) Priority Queue $\rightarrow$ if same priority

- Insertion $\rightarrow$ Front end $\rightarrow$ queue comes then use FIFO
- Deletion $\rightarrow$ Rear end $\rightarrow$ Array def (consecutive memory location)

(1) Ascending Priority Queue :-

"is a linear list of elements in which insertion takes place arbitrarily (randomly) but at the time of deletion you have to delete the smallest or minimum value."

$\rightarrow$ Insertion APQ ()
 $\rightarrow$ minDelete ()

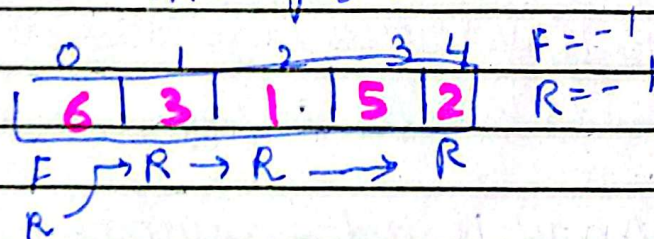
(2) Descending Priority Queue

"is a linear list of elements in which insertion occurs arbitrarily but at time of deletion you have to delete the largest or maximum value from descending priority Queue"

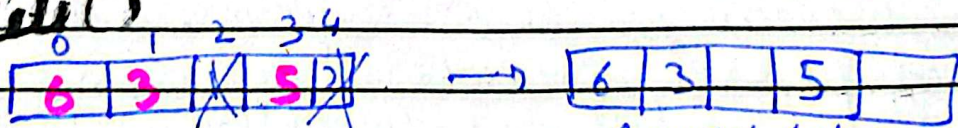
$\rightarrow$ Insertion APQ ()

Priority Queue

$\rightarrow$ Insertion DPQ ()
 $\rightarrow$ MaxDelete ()



$\rightarrow$ MinDelete ()



Note: Always we have to delete from end in this (not fulfilled, queue)

Q1: (perform) of 5. call insertion deletion time 5. 5.

1. Sort

Now looks like

Insertion — sort

X | 2 | 3 | 5 | 6

(Sort solution but takes more time, space, can generate overheads when same value repeats)

F → F

R

2 3 5 6 2

2 3 6 6

Sort

2. Swap function

Deletion

swap

• (Find Minimum)

• (Swap)

• (Delete)

Benefit of sort: when we delete, we can delete min value easily at 0 index.

• min values move to front.

(Swap smallest value with front and delete it)

Swap = Find min, swap, delete

3. Empty indicators (#, -1 etc)

overhead

6 | 2 | # | 5 | 2

Stack

• Array consecutive memory

can't delete from Front → Queue

(2) Insertion: overflow condition

In this case (in b/w values) are deleting.

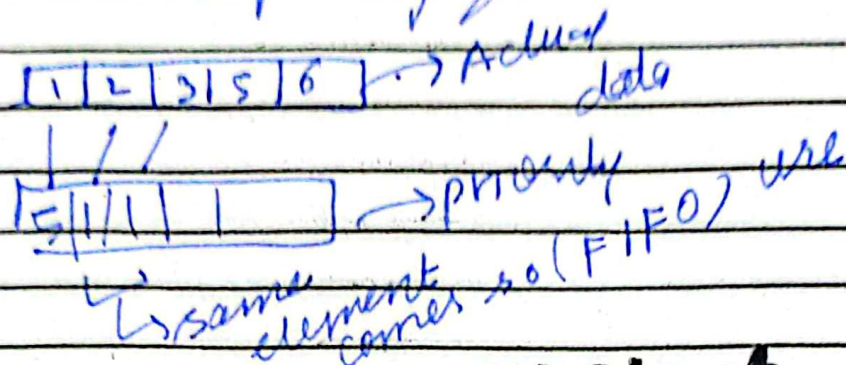
(3) we have to search at empty spaces also

Conclusion: Array is not a suitable solution for the Implementation of priority Queue

• we can do by linked list or heap tree

• Best solution of above = sort

- Atleast 2 queues are required to implement the priority queue



Time Complexity (1) Stack

constant time in stack

- constant Time $O(1)$ } both in push, pop
- ↳ every time you have to insert/delete from top of stack, no more further processing

(2) Queue: also provides constant time delete from front : constant time insert at rear $O(1)$

Applications of Queue

- Linear Queue - Not practically used
- Circular Queue eg = * Traffic control system
- * Network printer (multiple systems connected together, multiple command)
- Priority Queue : * Hospitals (emergency cases)
- * Downloading multiple files (we set priority of task to download)
- * Operating system (windows, LINUX, time sharing system).